

Task Scheduling on Multi-CPU Graph Architectures using Supervised Policy Learning

Louis Lejaille
louis.lejaille@epitech.eu

Abstract

Task scheduling on multi-CPU architectures is a complex combinatorial optimization problem, particularly when task dependencies are represented as directed acyclic graphs (DAGs). Traditional heuristic approaches can provide efficient scheduling decisions but often struggle to generalize across heterogeneous architectures and varying task graph topologies. In this work, we investigate a supervised learning approach based on a Multi-Layer Perceptron (MLP) trained through behavior cloning from a heuristic scheduling policy. System states are encoded into fixed-size normalized vectors to enable supervised learning of task-to-CPU allocation decisions. Experimental results show that the proposed model successfully reproduces heuristic behavior on known scheduling scenarios. However, significant performance degradation is observed on unseen task graphs, highlighting the limitations of flat vector representations and dense MLP architectures for graph-structured scheduling problems. These results motivate the exploration of graph-aware architectures such as Graph Neural Networks (GNNs) for improved structural generalization.

I. INTRODUCTION

Task scheduling on multiprocessor architectures has been an important research topic for several decades. In many modern computing systems, multiple processors can execute tasks simultaneously in order to improve overall performance and reduce application execution time.

However, in many architectures, tasks are not independent. Some operations require other tasks to be completed before execution can begin. These dependencies can be represented using Directed Acyclic Graphs (DAGs), where each node represents a task and each edge represents a dependency constraint.

This type of problem is commonly encountered in embedded systems, multi-core architectures, autonomous vehicles, drones, and distributed systems. In such environments, the objective is to determine which processor should execute each task in order to optimize resource utilization and minimize total execution time.

Classical approaches generally rely on heuristic methods. These heuristics use manually designed rules to select priority tasks or determine the most appropriate processors according to various criteria, such as processor workload, remaining dependencies, or hardware topology.

However, these methods present several limitations. Task graphs may vary significantly from one scenario to another, both in terms of structure and in the number of tasks, dependencies, or processor configurations involved. As a result, designing heuristics capable of generalizing efficiently to unseen configurations becomes increasingly difficult.

In this context, machine learning approaches appear as an interesting alternative. In particular, neural networks can learn scheduling strategies directly from data without requiring the explicit definition of complex rules for every new configuration.

In this study, we propose a supervised learning approach based on a Multi-Layer Perceptron (MLP). The model is trained through behavior cloning using scheduling decisions generated by a heuristic policy. System states are encoded as fixed-size vectors in order to enable supervised learning.

The obtained results show that this approach can successfully reproduce heuristic behavior on known scheduling scenarios. However, performance degrades significantly on unseen task graphs, highlighting the structural limitations of MLP architectures for scheduling problems involving dynamic graph structures. This study therefore exposes the limitations of flat vector representations and naturally motivates the exploration of graph-specialized architectures such as Graph Neural Networks (GNNs). [1]

II. RELATED WORK

A. Classical Scheduling Heuristics

Classical heuristic-based scheduling approaches were historically among the first solutions proposed for task scheduling problems on multiprocessor architectures. These methods rely on manually designed decision rules intended to optimize task allocation according to predefined criteria.

In task scheduling problems involving Directed Acyclic Graphs (DAGs), heuristic policies generally assign priority scores to tasks based on several characteristics, such as the number of dependencies, estimated execution time, processor availability, communication costs, or processor neighborhood occupancy. The scheduler then selects the task-processor pair associated with the highest priority score.

Several studies have also explored execution time prediction models in order to improve scheduling decisions, since accurately estimating task duration before execution remains a challenging problem in dynamic environments.[2]

Heuristic methods are generally computationally efficient and can produce strong results on environments exhibiting

similar structural patterns. However, task scheduling on graph-based architectures is fundamentally a combinatorial optimization problem. As the number of tasks, dependencies, and processors increases, the search space grows exponentially. Consequently, handcrafted heuristics often struggle to generalize across heterogeneous graph topologies and unseen scheduling configurations.

B. Reinforcement Learning Approaches

Reinforcement Learning (RL) has also been widely explored for scheduling optimization problems. [1] In RL-based approaches, an agent interacts with an environment by selecting actions and receiving reward signals that evaluate the quality of these actions. The objective of the model is to maximize the cumulative reward over time by progressively adapting its policy.

One of the major challenges in RL-based scheduling lies in reward design. Poorly designed reward functions may lead the agent to exploit unintended behaviors or converge toward locally optimal but globally inefficient policies. Therefore, defining stable and meaningful reward mechanisms is a critical aspect of reinforcement learning systems.[3]

Another important challenge concerns delayed rewards and reward engineering. In scheduling environments, decisions performed at a given time step may only impact the global execution performance much later during the scheduling process. This temporal dependency complicates the credit assignment problem, making it difficult for the agent to correctly associate rewards with past actions. Furthermore, poorly designed reward functions may lead the agent toward unintended optimization behaviors or locally optimal policies instead of globally efficient scheduling strategies.

Several reinforcement learning algorithms have been investigated for scheduling applications, including Deep Q-Networks (DQN), Proximal Policy Optimization (PPO)[1], and other policy-gradient-based approaches.

C. Supervised Learning Approaches

Supervised learning approaches aim to train a model from previously generated scheduling decisions. In this context, a heuristic policy is often used to generate labeled scheduling data, and the model learns to reproduce the behavior of this policy through behavior cloning.

This paradigm is commonly used in imitation learning applications, such as board games or autonomous systems, where expert demonstrations are available. The objective is not only to reproduce deterministic rules, but also to learn generalized decision patterns from observed examples.[4]

To improve generalization capabilities and avoid simple memorization of heuristic decisions, several approaches introduce stochasticity or noise during data generation and training. This allows the model to learn more robust scheduling behaviors under varying configurations.

Supervised learning methods generally provide faster training and more stable optimization than reinforcement learning approaches. However, their performance strongly depends on the quality and diversity of the heuristic-generated dataset.

D. Graph Neural Networks

Graph Neural Networks (GNNs) have recently emerged as a promising approach for scheduling problems involving graph-structured data.

In task scheduling environments, both task dependencies and processor communication topologies can naturally be represented as graphs. Unlike traditional dense neural networks such as Multi-Layer Perceptrons (MLPs), GNNs are specifically designed to exploit relational and structural information between nodes.

GNN-based approaches propagate information through graph connections in order to learn contextual representations of tasks and processors. This enables the model to better capture dependency structures, communication constraints, and topological relationships within the scheduling environment.

Several recent studies have demonstrated that GNNs provide improved generalization capabilities compared to flat vector representations, particularly when dealing with unseen graph structures or variable-sized scheduling problems.[5]

III. ENVIRONMENT DESIGN

A. Task Graph Representation

Tasks are represented using a Directed Acyclic Graph (DAG), a common representation for dependency-constrained scheduling problems. In this graph, each node corresponds to a task associated with an execution duration, while directed edges represent dependency constraints between tasks.

A task can only begin execution once all its predecessor tasks have been completed. Consequently, the scheduling process must respect the partial execution order imposed by the graph structure.

Each task is associated with a dynamic execution state. During the simulation, tasks can transition through several states, including pending, running, and completed. Initially, tasks without dependencies are directly executable, while dependent tasks remain blocked until all required predecessor tasks have finished execution.

B. CPU Graph Architecture

Processors are also represented as a graph structure in order to model communication topology between CPUs. Each processor node is connected to neighboring processors through communication links defining the underlying hardware architecture.

This topology plays an important role in scheduling decisions, since communication constraints and processor locality may influence execution efficiency. Modeling processors as a graph therefore allows the environment to better represent realistic multiprocessor architectures.

C. Environment Simulation

To study the scheduling problem, a discrete-time simulation environment was designed to reproduce task execution behavior on graph-based CPU architectures.

Each environment is generated from a deterministic seed in order to guarantee reproducibility across experiments. This

reproducibility is essential for comparing the behavior of different scheduling approaches, such as heuristic policies and neural network models, under identical scenarios.

For each generated environment, a maximum number of tasks and processors is defined. The simulation then generates both the task dependency graph and the CPU topology graph according to the selected seed.

Processor nodes maintain execution-related information, including currently assigned tasks and remaining execution time. Task nodes contain information such as execution duration, dependency lists, dependent tasks, and execution status.

Once the environment is initialized, a scheduling scenario is executed until all tasks have been completed. The simulation evolves through discrete ticks using a step-based update mechanism. At each tick, running tasks progress toward completion, processor states are updated, and newly executable tasks may become available for scheduling.

D. Action Space

At each simulation step, the scheduler selects an action from the available action space. An action consists either of assigning a task to an available processor or selecting a WAIT action when no valid scheduling decision can currently be performed.

Task assignment actions are constrained by both task dependencies and processor availability. Only executable tasks can be assigned, and only idle processors may receive new tasks.

Once all tasks have been completed, the scenario terminates and the total execution time, commonly referred to as the makespan, is measured as the primary performance metric.

E. Environment Rendering and Visualization

The simulation environment also includes a human-readable rendering mode used for visualization and debugging purposes.

This rendering system allows visualization of both task dependency graphs and processor activity throughout execution. It provides an interpretable representation of task progression, processor utilization, and scheduling decisions over time.

Such visualization tools are particularly useful for validating scheduler behavior, analyzing execution dynamics, and facilitating experimental debugging. Figure [1] illustrates an example of the rendering interface used during simulation experiments.

IV. DATASET GENERATION

A. Heuristic Policy

In order to train a supervised learning model, a heuristic scheduling policy was designed to generate labeled scheduling decisions. This heuristic acts as a teacher policy and provides reference actions used for behavior cloning.

The heuristic operates by evaluating both executable tasks and available processors according to predefined scoring functions.

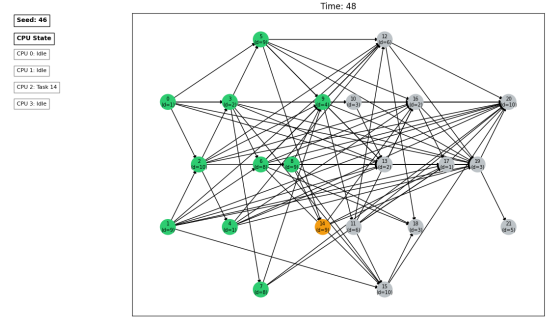


Fig. 1. Visualization of the scheduling environment during execution.

For task selection, each ready task is assigned a priority score based on structural and execution-related characteristics. The scoring function combines the number of dependent tasks and the execution duration of the task. Tasks with a larger number of dependents are prioritized in order to accelerate global graph progression, while longer tasks are also favored to reduce potential execution bottlenecks.

For processor selection, available CPUs are evaluated according to local processor occupancy. More specifically, processors with fewer busy neighboring CPUs are preferred in order to reduce local communication congestion and improve resource distribution across the processor graph.

The final scheduling decision is obtained by selecting the task and processor pair associated with the highest heuristic scores. A small amount of random noise is additionally introduced during scoring in order to diversify generated scheduling trajectories and reduce deterministic behavior during dataset generation.

B. Behavior Cloning

The proposed learning framework follows a supervised imitation learning paradigm commonly referred to as behavior cloning.

Instead of directly learning through reinforcement signals, the model learns to imitate the decisions produced by the heuristic scheduling policy. For each environment state, the heuristic generates the action considered optimal according to its scoring strategy. The neural network is then trained to reproduce these scheduling decisions from encoded environment states.

This approach transforms the scheduling problem into a supervised classification problem, significantly simplifying the training process compared to reinforcement learning approaches. In particular, behavior cloning avoids reward engineering difficulties and provides more stable optimization dynamics during training.

However, the learned policy remains strongly dependent on the quality and limitations of the heuristic used during dataset generation. Consequently, the model may inherit heuristic

biases and struggle to exceed the performance of the original teacher policy.

C. Dataset Construction

The dataset is generated by simulating multiple scheduling environments using different random seeds. Each seed produces a distinct task graph and CPU topology configuration, allowing the generation of diverse scheduling scenarios.

For every simulation step, the current environment state is captured before applying the heuristic decision. The heuristic policy then computes the scheduling action associated with the current state, and the resulting state-action pair is stored in the dataset.

This process is repeated until all tasks within the environment have been completed. Across all generated environments, the final dataset therefore consists of a collection of encoded environment states associated with heuristic scheduling decisions.

D. State Normalization

To improve training stability and maintain consistent feature scales across the dataset, all numerical features are normalized before being provided to the neural network.

Task durations, dependency counts, processor neighborhood sizes, and remaining execution times are normalized using predefined maximum environment constraints. This normalization prevents feature magnitude imbalance and facilitates neural network optimization during training.

Categorical task states are converted into numerical representations before normalization in order to produce fully numerical input vectors compatible with dense neural network architectures.

E. State Encoding

Since Multi-Layer Perceptrons require fixed-size numerical inputs, the graph-structured environment must be transformed into a fixed-dimensional vector representation.

However, both task graphs and processor graphs may vary in size between environments. To address this issue, maximum numbers of tasks and processors are predefined, and padding is applied whenever an environment contains fewer elements than these limits.

Each task is encoded using several normalized features, including execution duration, execution state, dependency count, and dependent count. Similarly, each processor is encoded using processor availability, remaining execution time, and neighborhood size.

The final environment representation is obtained by concatenating all task features and processor features into a single flattened vector representation:

$$X = [\text{task_features} \parallel \text{cpu_features}]$$

This encoding strategy enables the use of standard dense neural networks such as MLPs. However, flattening graph-structured information into fixed-size vectors introduces structural limitations. In particular, relational information and graph

topology are only implicitly represented, which may negatively impact generalization on unseen graph configurations.

V. MLP ARCHITECTURE

A. Overview

The Multi-Layer Perceptron (MLP) is one of the foundational architectures in artificial neural networks. Although the recent rise of artificial intelligence has largely popularized deep learning for the general public, MLP-based architectures have been studied and used for several decades in a wide variety of machine learning applications [6].

An MLP consists of a sequence of fully connected layers organized into three main components:

- an input layer,
- one or several hidden layers,
- and an output layer.

As described in previous studies, MLP architectures are composed of interconnected neurons where each layer performs successive nonlinear transformations on the input data [6].

In the proposed approach, the input layer receives the fixed-size vector representation of the scheduling environment described in the previous section. Hidden layers progressively transform these input features into higher-level latent representations, while the output layer predicts the scheduling action to perform.

The architecture is trained using supervised learning in order to imitate the heuristic scheduling policy through behavior cloning.

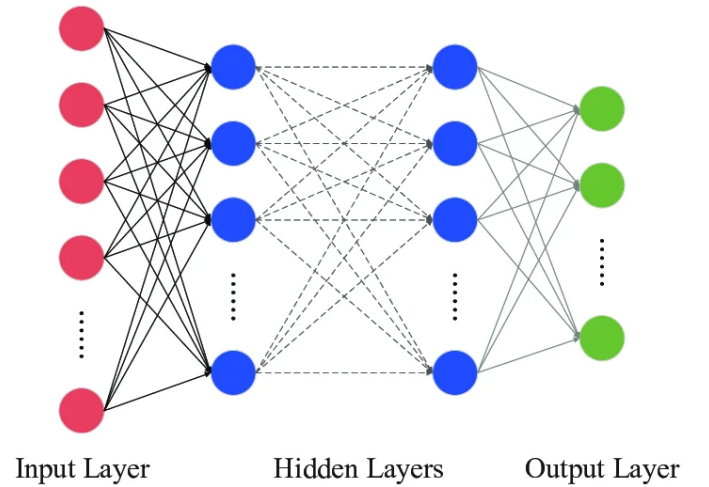


Fig. 2. MLP architecture used for scheduling policy learning

B. Forward Propagation

Each neuron within the network computes a weighted sum of its inputs followed by a nonlinear activation function. For a given layer l , the neuron activations are computed as:

$$z^{(l)} = W^{(l)}x^{(l-1)} + b^{(l)}$$

$$x^{(l)} = f(z^{(l)})$$

where:

- $W^{(l)}$ represents the weight matrix of layer l ,
- $b^{(l)}$ represents the bias vector,
- $f(\cdot)$ is the activation function,
- $x^{(l-1)}$ corresponds to the input of the current layer.

Through successive transformations across hidden layers, the network produces an output vector representing the probability distribution over possible scheduling actions.

MLP architectures are known to become increasingly computationally expensive as the number of layers and neurons increases [6]. The computational complexity of the network grows proportionally with the number of hidden neurons and inter-layer connections.

C. Output Layer and Action Prediction

The output layer contains one neuron for each possible scheduling action within the discrete action space defined previously.

The network output is transformed into a probability distribution using the Softmax function:

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

where:

- K represents the number of possible actions,
- z_i is the output logit associated with action i .

The predicted action corresponds to the action with the highest predicted probability.

D. Loss Function

Since the scheduling problem is formulated as a multiclass classification problem, the network is trained using the Cross-Entropy loss function.

For a single training example, the loss is defined as:

$$\mathcal{L} = - \sum_{i=1}^K y_i \log(\hat{y}_i)$$

where:

- y_i is the ground-truth target distribution,
- $\hat{y}_i$ is the predicted probability for action i .

The objective of the optimization process is to minimize this loss over the entire training dataset.

E. Backpropagation and Optimization

During training, the network performs two successive phases:

- forward propagation,
- backward propagation.

During forward propagation, input data passes through the network in order to produce action predictions. The prediction error is then measured using the loss function.

The error gradients are subsequently propagated backward through the network using the backpropagation algorithm. This process computes the contribution of each parameter to the prediction error using the chain rule of differentiation.

Network parameters are then updated using the Adam optimization algorithm, which combines momentum-based optimization and adaptive learning rates in order to improve convergence stability and training efficiency.

The parameter update rule can be summarized as:

$$\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}}{\partial \theta}$$

where:

- θ represents the model parameters,
- α is the learning rate,
- $\frac{\partial \mathcal{L}}{\partial \theta}$ represents the computed gradients.

F. Training and Evaluation Phases

The proposed workflow is divided into two main stages:

- a training phase,
- and an evaluation phase.

During training, the model receives encoded environment states together with heuristic-generated target actions. The network progressively adjusts its parameters in order to minimize prediction error and imitate the heuristic policy.

Once training is completed, the model enters the evaluation phase. During this stage, the network independently predicts scheduling actions without access to heuristic supervision. The resulting scheduling performance can then be compared to the original heuristic policy in order to evaluate the model's generalization capabilities.

VI. EXPERIMENTAL RESULTS

A. Experimental Setup

The experimental pipeline was divided into three main stages:

- environment generation,
- dataset construction,
- model training and evaluation.

The dataset was generated by simulating multiple scheduling scenarios using deterministic random seeds. Each generated environment produced a unique task graph and CPU topology configuration. The simulation framework was organized into separate modules for environment simulation, dataset generation, training, evaluation, and result analysis.

Two different dataset configurations were investigated during the experiments:

- a large-scale dataset containing 5000 generated scheduling scenarios,
- and a reduced dataset containing 1000 scenarios.

The MLP architecture used during training received 175 input features corresponding to the flattened and normalized environment representation described previously.

The network architecture was composed of:

- an input layer of 175 neurons,

- two hidden layers of 256 neurons,
- ReLU activation functions,
- and an output layer corresponding to the discrete scheduling action space.

The model was trained using the Cross-Entropy loss function together with the Adam optimizer using a learning rate of 0.001.

Training followed the standard supervised learning workflow:

- forward propagation,
- loss computation,
- backpropagation,
- parameter update.

The objective of the optimization process was to minimize prediction error between heuristic-generated scheduling actions and model predictions.

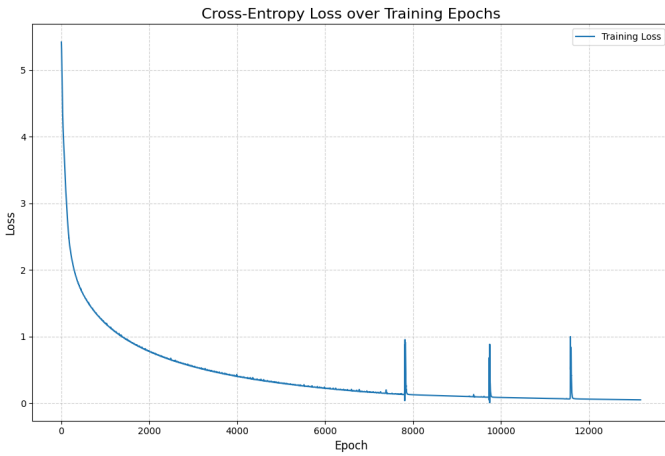


Fig. 3. Cross-entropy training loss during supervised learning. The model progressively converges toward a stable low-loss regime despite occasional transient spikes caused by difficult scheduling states.

B. Initial Training Results and WAIT Issue

The first experiments were conducted using the complete dataset generated from 5000 scheduling scenarios. In this initial version of the dataset, WAIT actions were recorded whenever no valid scheduling decision could be performed.

Evaluation results showed a strong degradation in scheduling performance compared to the heuristic baseline.

TABLE I
INITIAL EVALUATION RESULTS

Method	Average Total Time
MLP	122.75
Heuristic	71.05

Detailed analysis revealed that the model excessively predicted the WAIT action during inference. In many scenarios, the scheduler repeatedly selected invalid or already completed task assignments, leading to degenerate scheduling behavior dominated by WAIT actions.

This issue was primarily caused by dataset imbalance. Since WAIT actions appeared very frequently during simulation, the model progressively converged toward overpredicting this action in order to minimize the Cross-Entropy loss.

In some evaluation scenarios, the model entered nearly infinite WAIT loops. To prevent non-terminating simulations, a maximum execution limit of 200 ticks was introduced.

C. Dataset Filtering and Performance Improvement

To address the WAIT action imbalance issue, the dataset generation pipeline was modified.

Instead of recording WAIT actions whenever no scheduling decision was possible, the simulation continued advancing internally without storing these intermediate WAIT states inside the dataset. This significantly reduced the proportion of WAIT actions and improved class balance during training.

Additionally, the number of generated scenarios was reduced from 5000 to 1000 environments. This reduction limited structural diversity and simplified the learning distribution.

After applying these modifications, the obtained results became significantly closer to the heuristic baseline.

TABLE II
EVALUATION RESULTS AFTER WAIT FILTERING

Method	Average Total Time
MLP Filtered	71.15
Heuristic	71.05

The filtered MLP successfully reproduced the heuristic policy on known scheduling distributions. Training convergence also became significantly faster, with the loss rapidly decreasing below 0.05.

D. Evaluation on Known Scheduling Scenarios

The filtered model was evaluated on a larger set of scheduling scenarios generated from seeds ranging from 142 to 192. Figure 4 compares the heuristic scheduler and the trained MLP model on these scenarios.

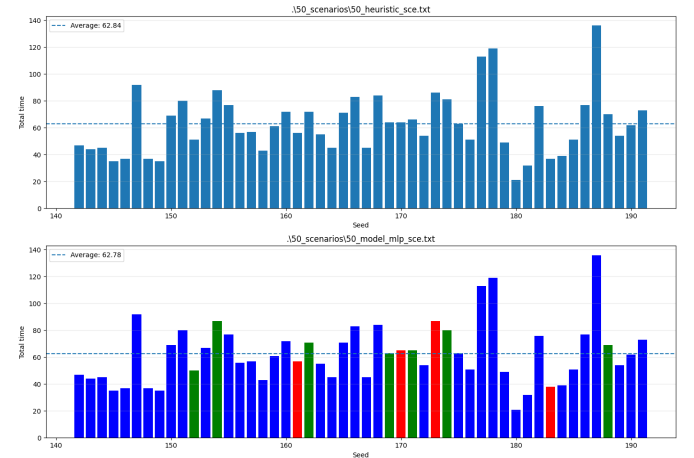


Fig. 4. Comparison between the heuristic scheduler and the filtered MLP model on known scheduling scenarios.

The obtained results were highly similar between both approaches:

- Heuristic average total time: 62.84
- MLP average total time: 62.78

The minimum and maximum execution times were also nearly identical across both methods. In some scenarios, the MLP slightly outperformed the heuristic policy, while in others it produced slightly longer execution times.

These observations indicate that the MLP successfully learned local scheduling patterns from the heuristic-generated dataset and was capable of reproducing similar scheduling decisions on environments belonging to comparable distributions.

E. Generalization on Unseen Scenarios

Although the filtered MLP produced strong results on known scheduling distributions, generalization performance significantly degraded on unseen environments.

To evaluate structural generalization capabilities, the model was tested on a new set of scheduling scenarios generated from previously unseen seeds and graph configurations. Figure 5 presents the comparison between the heuristic scheduler and the MLP model on these unseen environments.

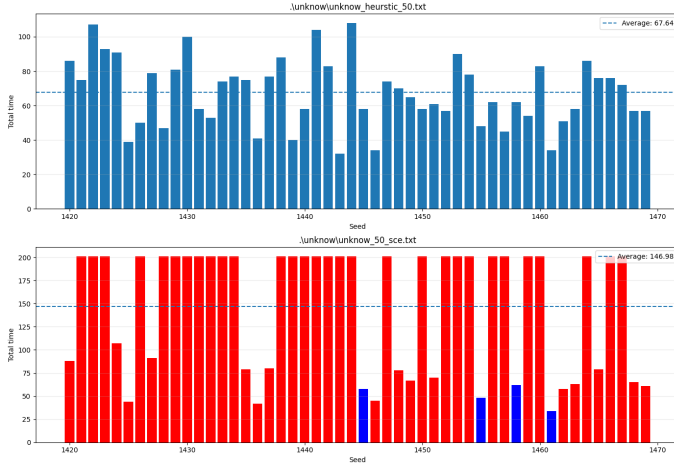


Fig. 5. Comparison between the heuristic scheduler and the MLP model on unseen scheduling scenarios.

The performance degradation became extremely significant on these unseen graph structures.

$$\text{Average Total Time} = 146.98$$

Compared to the heuristic baseline, the trained MLP frequently produced unstable scheduling policies and failed to correctly adapt to new graph topologies and dependency structures.

These results highlight the limitations of dense MLP architectures for graph-structured scheduling problems. Although the model was capable of reproducing heuristic behavior on known distributions, it struggled to generalize to structurally

different environments containing unseen task dependencies and processor configurations.

VII. LIMITATIONS

Although the proposed MLP-based approach successfully reproduced heuristic scheduling behavior on known scheduling distributions, several important limitations were identified during experimentation.

A. Loss of Graph Structure

One of the primary limitations of the proposed approach comes from the flattening process used to encode graph-structured environments into fixed-size vectors.

Task dependencies and processor communication topologies are naturally represented as graphs containing relational and structural information. However, in order to make the environment compatible with a dense MLP architecture, both task and CPU graphs were transformed into flattened numerical vectors through concatenation.

This transformation removes explicit graph connectivity information and forces the network to implicitly infer structural relationships only from positional feature encoding. As a result, the model struggles to correctly capture long-range dependencies, graph topology, and relational interactions between tasks and processors.

The experimental results strongly support this limitation. While the MLP successfully reproduced heuristic behavior on known scheduling scenarios, performance significantly degraded on unseen graph configurations.

B. Permutation Sensitivity

Another important limitation of dense MLP architectures is permutation sensitivity.

In graph-based scheduling problems, the ordering of tasks or processors inside the encoded vector should ideally not affect the scheduling strategy. However, because MLPs process fixed positional inputs, changing the ordering of tasks or CPUs inside the flattened representation may produce completely different activations, even when the underlying graph structure remains identical.

Consequently, the model becomes highly dependent on the exact encoding order used during dataset generation. This sensitivity limits the ability of the network to learn generalized relational reasoning independently from node ordering.

C. Poor Structural Generalization

The obtained experimental results demonstrate that the proposed MLP architecture exhibits poor structural generalization capabilities.

When evaluated on scheduling scenarios generated from distributions similar to the training dataset, the model achieved performances nearly identical to the heuristic policy. However, when exposed to previously unseen graph structures, the scheduling policy became unstable and performance deteriorated drastically.

The average execution time increased significantly on unseen environments, revealing that the model had primarily

memorized local scheduling patterns rather than learning robust graph-aware scheduling strategies.

This behavior indicates that dense MLP architectures are poorly suited for environments involving:

- variable graph topologies,
- dynamic dependency structures,
- heterogeneous processor configurations,
- and large combinatorial scheduling spaces.

D. Dynamic Action Space Constraints

Another limitation concerns the action space representation.

In realistic scheduling problems, the set of valid actions dynamically changes during execution depending on:

- task dependencies,
- processor availability,
- and current execution states.

However, the proposed approach models scheduling decisions using a fixed discrete action space. All possible task-processor pairs are encoded into a static classification problem regardless of whether actions are currently valid or invalid.

This formulation introduces additional learning complexity, since the model must implicitly learn action validity constraints during training. As observed during the initial experiments, this limitation contributed to the overprediction of invalid actions and excessive WAIT behaviors.

More generally, the fixed-action formulation does not naturally scale to larger environments containing varying numbers of tasks and processors.

VIII. FUTURE WORK

The limitations observed during experimentation naturally motivate the exploration of graph-specialized neural architectures for future scheduling research.

A promising direction consists of replacing dense Multi-Layer Perceptrons with Graph Neural Networks (GNNs). Unlike traditional MLP architectures, GNNs are specifically designed to operate directly on graph-structured data while preserving relational and topological information.

In graph-based scheduling problems, both task dependencies and processor communication structures are naturally represented as graphs. GNN architectures can therefore exploit these structures directly instead of relying on flattened vector representations.

One of the key mechanisms of GNNs is message passing. During this process, nodes iteratively exchange information with their neighboring nodes in order to progressively build contextual representations of the graph structure.

For a task scheduling problem, this mechanism would allow tasks to dynamically aggregate information about:

- their dependencies,
- neighboring tasks,
- processor states,
- communication constraints,
- and global execution context.

This relational reasoning capability could significantly improve the model's ability to understand scheduling constraints and capture long-range structural dependencies.

Another important advantage of GNNs lies in their ability to generate node embeddings. Instead of encoding the entire environment into a single flattened vector, each node learns its own latent representation based on both local and global graph information.

Such representations are generally more robust to:

- varying graph sizes,
- dynamic topologies,
- and unseen scheduling structures.

Consequently, GNN-based approaches may provide significantly improved structural generalization compared to dense MLP architectures.

Future work could also investigate reinforcement learning approaches combined with graph neural networks in order to directly optimize scheduling policies through interaction with the environment rather than pure imitation learning.

More advanced architectures such as Graph Attention Networks (GATs), Pointer Networks, or Graph Transformers may also represent promising research directions for large-scale combinatorial scheduling problems.

Overall, the results obtained in this study suggest that preserving graph topology and relational information is essential for developing robust scheduling models capable of generalizing across heterogeneous multiprocessor environments.

IX. CONCLUSION

In this study, we investigated a supervised learning approach for task scheduling on multiprocessor graph architectures using a Multi-Layer Perceptron (MLP) trained through behavior cloning.

To address this problem, a complete simulation environment was designed, including task dependency graphs, CPU communication topologies, scheduling dynamics, and heuristic-based dataset generation. Environment states were encoded into fixed-size normalized vectors in order to train a dense neural network capable of predicting scheduling decisions.

Experimental results demonstrated that the proposed approach was able to successfully reproduce heuristic scheduling behavior on known scheduling distributions. In particular, filtering excessive WAIT actions during dataset generation significantly improved training stability and scheduling performance.

However, the obtained results also revealed important structural limitations of dense MLP architectures for graph-based scheduling problems. Although the model performed well on environments similar to those seen during training, generalization capabilities degraded significantly on unseen graph structures and scheduling configurations.

These observations highlight the limitations of flattened vector representations for relational and topological reasoning tasks. More generally, the study suggests that preserving graph structure information is essential for developing robust

scheduling models capable of generalizing across heterogeneous environments.

Future work will therefore focus on graph-specialized architectures such as Graph Neural Networks (GNNs), which naturally preserve relational information and may provide improved structural generalization for dynamic scheduling problems.

REFERENCES

- [1] J. Park, "Learning to schedule job-shop problems: Representation and policy learning using graph neural network and reinforcement learning," *International Journal of Production Research*, 2021.
- [2] E. Sherman, "Using machine learning to improve job scheduling in datacenters," in *Proceedings of an Academic Conference*.
- [3] D. Dewey, "Reinforcement learning and the reward engineering principle," *Future of Humanity Institute Faculty of Philosophy University of Oxford*, 2014.
- [4] H. VUONG, "An investigative study about application of supervised learning for making predictions in chess," *Degree project in computer engineering, first cycle, 15 credits Stockholm, SWEDEN*, 2017.
- [5] K. T. Chung, "Neural combinatorial optimization with reinforcement learning in industrial engineering: A survey," *Artificial Intelligence Review*, 2024.
- [6] K. Y. Chan, "Deep neural networks in the cloud: Review, applications, challenges and research directions," *Elsevier*, 2023.
- [7] J. Park, "Strait: Perceiving priority and interference in ml inference serving," *arXiv preprint arXiv:2604.28175*, 2026. [Online]. Available: <https://arxiv.org/pdf/2604.28175>
- [8] A. HATTAY, "On the role of dag topology in energy-aware cloud scheduling: A gnn-based deep reinforcement learning approach," *arXiv preprint arXiv:2604.09202*, 2026.
- [9] S. Biswas, "A machine learning approach for predicting efficient cpu scheduling algorithm," *ResearchGate preprint*, 2024.
- [10] S. Feng, "Using mlp networks to design a production scheduling system," *Computers & Operations Research*, 2002.
- [11] W. Sun, "Breaking multilayer perceptron limitations for traffic flow forecasting with structured patch learning," *Engineering Applications of Artificial Intelligence*, 2026.